Tests

Patterns and Anti-Patterns

Yegor Bugayenko

Lecture #14 out of 16
80 minutes

The slidedeck was presented by the author in this YouTube Video.

All visual and text materials presented in this slidedeck are either originally made by the author or taken from public Internet sources, such as web sites. Copyright belongs to their respected authors.

- Twenty Three Test Anti-Patterns
- Mocking Frameworks
- OOP + Tests
- Testable Code
- Books, Venues, Call-to-Action

Chapter #1:

Twenty Three Test Anti-Patterns



https://www.yegor256.com/2018/12/11/unit-testing-anti-patterns.html →

Cuckoo
Test-per-Method
Anal Probe
Conjoined Twins
Happy Path
Slow Poke
Giant
Mockery
Inspector
Generous Leftovers
Local Hero
Nitpicker
Secret Catcher
Dodger
Loudmouth
Greedy Catcher
Sequencer
Enumerator
Free Ride
Excessive Setup
Line hitter
Forty-Foot Pole Test
The Liar

Test-per-Method

Although a one-to-one relationship between test and production classes is a reasonable starting point, a one-to-one relationship between test and production method is almost always a bad idea.

Anal Probe

A test that has to use unhealthy ways to perform its task, such as reading private fields using reflection.

Happy Path

The tests stay on happy paths (i.e. expected results, e.g. 18 years old) without testing for boundaries and exceptions (e.g. -2 years old)

Slow Poke

A unit test that runs incredibly slow. When developers kick it off, they have time to go to the bathroom, grab a smoke, or worse, kick the test off before they go home at the end of the day.

Giant

A unit test that, although it is validly testing the object under test, can span thousands of lines and contain many many test cases. This can be an indicator that the system-under-test is a God Object.

Mockery

Sometimes mocking can be good, and handy. But sometimes developers can lose themselves in their effort to mock out what isn't being tested. In this case, a unit test contains so many mocks, stubs, and/or fakes that the system under test isn't even being tested at all, instead data returned from mocks is what is being tested.

Inspector

A unit test that violates encapsulation in an effort to achieve 100% code coverage, but knows so much about what is going on in the object that any attempt to refactor will break the existing test and require any change to be reflected in the unit test.

Generous Leftovers (aka Chain Gang, Wet Floor)

An instance where one unit test creates data that is persisted somewhere, and another test reuses the data for its own devious purposes. If the “generator” is ran afterward, or not at all, the test using that data will outright fail.

Local Hero (aka Hidden Dependency)

A test case that is dependent on something specific to the development environment it was written on, in order to run. The result is that the test passes on development boxes, but fails when someone attempts to run it elsewhere.

Dodger

A unit test which has lots of tests for minor (and presumably easy to test) side effects, but never tests the core desired behavior. Sometimes you may find this in database access related tests, where a method is called, then the test selects from the database and runs assertions against the result.

Loudmouth

A unit test (or test suite) that clutters up the console with diagnostic messages, logging, and other miscellaneous chatter, even when tests are passing.

Sequencer

A unit test that depends on items in an unordered list appearing in the same order during assertions.

Enumerator (aka Test With No Name)

Unit tests where each test case method name is only an enumeration, e.g. test1, test2, test3. As a result, the intention of the test case is unclear, and the only way to be sure is to read the test case code and pray for clarity.

Free Ride (aka Piggyback)

Rather than write a new test case method to test another feature or functionality, a new assertion rides along in an existing test case.

Excessive Setup (aka Mother Hen)

A test that requires a lot of work to set up in order to even begin testing. Sometimes several hundred lines of code are used to setup the environment for one test, with several objects involved, which can make it difficult to really ascertain what is being tested due to the “noise” of all of the setup.

Line hitter

At first glance, the tests cover everything and code coverage tools confirm it with 100%, but in reality the tests merely hit the code, without doing any output analysis.

The Liar (aka Evergreen Tests)

A test doesn't validate any behaviour and passes in every scenario. Any new bug introduced in the code will never be discovered by this test. It was probably created after the implementation was finished, so the author of this test couldn't know whether the test actually tests something.

Chapter #2:

Mocking Frameworks

Mocking Frameworks are Evil

https://www.youtube.com/watch?v=1bAixLaOCSA →



Instead: Doubles or Fake Objects



https://www.youtube.com/watch?v=CCr8hRE_TUs →

```
class Book {
  private int id;
  private Database database;
  Book(Database db, int i) {
    this.database = db;
    this.id = i;
  }
  String title() {
    return this.database.fetch(
      “SELECT title FROM book WHERE id=?”,
      this.id
    );
  }
}
```

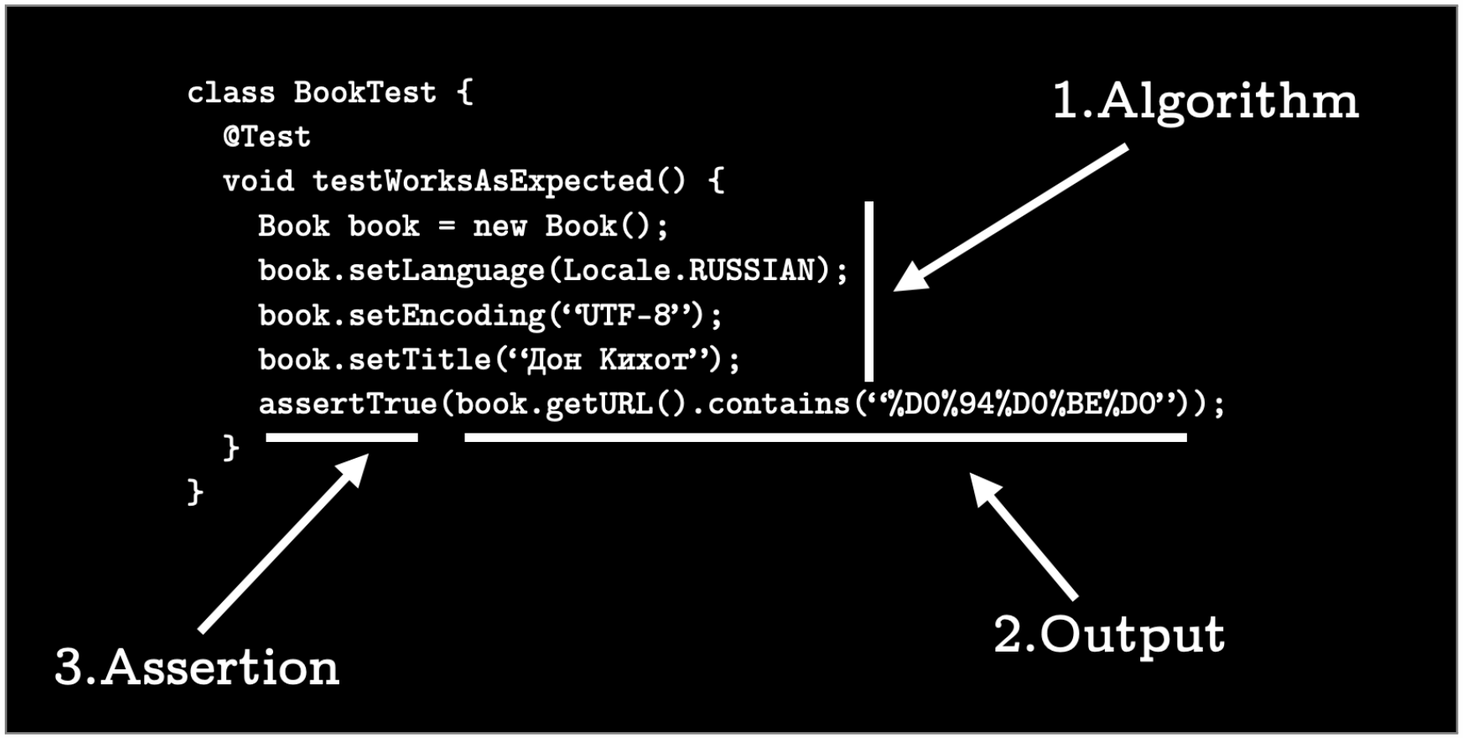
```
class BookTest {
  @Test
  public canFetchTitleFromDatabase() {
    Book book = new Book(
      new FakeDatabase(), 123
    );
    assert book.title().equals(“UML Distilled”);
  }
}
```

Chapter #3:

OOP + Tests

xUnit Tests are Procedural

https://www.youtube.com/watch?v=CCr8hRE_TUs →



Instead: Single-Method Unit Tests

https://www.youtube.com/watch?v=CCr8hRE_TUs →

```
class BookTest {
  @Test
  void testWorksAsExpected() {
    // nothing goes here
    assertThat(
      new Book("Дон Кихот", "UTF-8", Locale.RUSSIAN),
      new HasURL(new StringContains("%D0%94%D0%BE%D0"))
    );
  }
}
```

Chapter #4:

Testable Code

All Dependencies Are Injectable

Wrong

```
1 import java.nio.file.Files;
2 class Book {
3   String title() {
4     return Files.readAllLines(
5       Paths.get("/my-files/book.txt
  ")
6     )[0];
7   }
8 }
```

Right

```
1  import java.nio.file.Files;
2  class Book {
3    private Path file;
4    Book(Path f) {
5      this.file = f;
6    }
7    String title() {
8      return Files.readAllLines(
9        this.file
10     )[0];
11   }
12 }
```

Each Interface Has a Fake Object

Interface

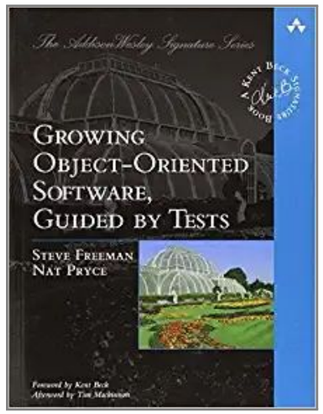
```
1 interface Book {
2   String title();
3   String author();
4 }
```

Fake Book

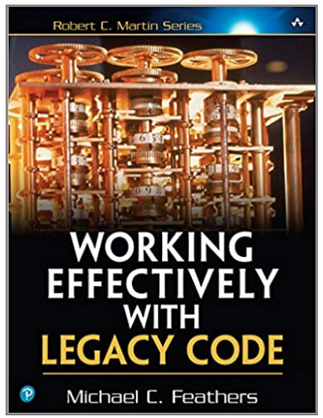
```
1  class FakeBook implements Book {
2    @Override
3    String title() {
4      return "Fake Title";
5    }
6    @Override
7    String author() {
8      return "Fake Author";
9    }
10 }
```

Chapter #5:

Books, Venues, Call-to-Action



Steve Freeman and Nat Pryce. *Growing Object-Oriented Software, Guided by Tests*. Pearson Education, 2009. DOI: 10.5555/1655852



Michael Feathers. *Working Effectively With Legacy Code*. Prentice Hall, 2004. DOI: 10.5555/1050933

Where to go:

International Symposium on Software Testing and Analysis (ISSTA)

Call to Action:

Get rid of mocking framework in your code and only use fake objects.

Still unresolved issues:

- How to detect test anti-patterns automatically?
- How to refactor tests automatically?
- How to write fake objects faster?
- How to educate programmers to write better tests?

References

[1] Michael Feathers. *Working Effectively With Legacy Code*. Prentice Hall, 2004. DOI: 10.5555/1050933.

[2] Steve Freeman and Nat Pryce. *Growing Object-Oriented Software, Guided by Tests*. Pearson Education, 2009. DOI: 10.5555/1655852.